

Transistor-Level Design and Simulation of a 4×4 6T SRAM Array Using LTspice

1. Project Objective

The purpose of this project was to understand how transistor-level CMOS circuits can be developed into a functional static random-access memory (SRAM) structure. The project began with MOSFET and CMOS fundamentals and progressed toward the design and simulation of a 6T SRAM cell and a 4×4 SRAM array using LTspice.

The main goals were to:

- Understand the operation of NMOS and PMOS transistors in CMOS circuits.
 - Build and analyze cross-coupled CMOS inverters.
 - Design a transistor-level 6T SRAM cell.
 - Verify write, hold, and read behavior.
 - Expand the single SRAM cell into 2×2 and 4×4 memory arrays.
 - Verify row-selective multi-bit storage and data retention.
-

2. Background

A standard 6T SRAM cell consists of six MOSFETs. Four transistors form two cross-coupled CMOS inverters, while two additional NMOS transistors act as access transistors.

The two cross-coupled inverters create two stable states:

- $Q = 1$ and $QB = 0$
- $Q = 0$ and $QB = 1$

These two states represent one stored binary bit.

The access transistors connect the internal storage nodes to the differential bitlines BL and BLB when the wordline WL is enabled.

Unlike DRAM, SRAM does not require periodic refresh while power remains applied. SRAM is commonly used in high-speed on-chip memories such as processor cache and local memory blocks.

3. Single 6T SRAM Cell Design

The SRAM cell was constructed using:

- Two NMOS pull-down transistors
- Two PMOS pull-up transistors
- Two NMOS access transistors
- Differential bitlines BL and BLB
- One wordline WL
- Internal storage nodes Q and QB

Custom MOSFET models, NM_SRAM and PM_SRAM, were used for transistor-level simulation in LTspice.

The cross-coupled inverter structure provides positive feedback. Once one internal node becomes HIGH and the opposite node becomes LOW, the feedback maintains the stored state while power remains applied.

Figure 1. Transistor-level schematic of the 6T SRAM cell.

[Insert clean screenshot of the final SRAM_Cell schematic here.]

The figure should show M1–M6, Q, QB, BL, BLB, WL, and VDD clearly.

4. Single-Cell Write and Hold Verification

To verify the write operation, complementary signals were applied to BL and BLB while WL was enabled.

For example, a logic 1 was written using:

- BL = 1.2 V
- BLB = 0 V
- WL = HIGH

During the write operation, Q transitioned toward 1.2 V and QB transitioned toward 0 V. After WL returned LOW, the state remained stable.

The opposite bitline condition was also used to verify writing logic 0.

Figure 2. Single-cell write and hold transient simulation.

[Insert the waveform showing WL, Q, and QB here.]

The important result to highlight is that Q and QB change while WL is active and remain stable after WL returns LOW.

5. Hierarchical SRAM Cell

After verifying the individual SRAM cell, the transistor-level circuit was converted into a reusable LTspice hierarchical symbol.

The hierarchical cell exposed four external connections:

- VDD
- WL
- BL
- BLB

The internal storage nodes Q and QB remained inside the SRAM cell.

This approach allowed the same verified 6T SRAM design to be reused when constructing larger memory arrays without manually redrawing the six-transistor circuit for every cell.

Figure 3. Hierarchical SRAM cell symbol used for array construction.

[Insert screenshot of the SRAM_Cell symbol here.]

6. 2×2 SRAM Array

Four hierarchical SRAM cells were first connected to form a 2×2 array.

The array contained:

- Two wordlines: WL0 and WL1
- Two differential bitline pairs:
 - BL0 / BLB0
 - BL1 / BLB1

Cells located in the same row shared a wordline. Cells located in the same column shared a differential bitline pair.

The array was used as an intermediate validation step before constructing the final 4×4 array.

The tested data pattern was:

Row	Stored Data
-----	-------------

Row 0	10
-------	----

Row 1	01
-------	----

Writing Row 1 did not alter the value previously stored in Row 0, demonstrating row isolation.

Figure 4. 2×2 SRAM array architecture and write verification.

[Insert either the 2×2 schematic or the most representative 2×2 waveform here.]

7. 4×4 SRAM Array Design

After validating the 2×2 structure, the design was expanded into a 4×4 SRAM array containing 16 SRAM cells.

The array used four wordlines:

- WL0
- WL1
- WL2
- WL3

and four differential bitline pairs:

- BL0 / BLB0
- BL1 / BLB1
- BL2 / BLB2
- BL3 / BLB3

Each wordline selected one row containing four SRAM cells. Therefore, each row stored one four-bit word.

Figure 5. Final 4×4 SRAM array schematic containing 16 hierarchical 6T SRAM cells.

[Insert the clean final 4×4 LTspice schematic here.]

The intended stored words were:

Row	Stored Data
-----	-------------

Row 0	1010
-------	------

Row 1 0110

Row 2 1100

Row 3 0011

The wordlines were activated sequentially:

- WL0: approximately 1–3 ns
- WL1: approximately 5–7 ns
- WL2: approximately 9–11 ns
- WL3: approximately 13–15 ns

Piecewise-linear voltage sources were used on the bitlines so that the data presented to the array changed before each row was selected.

8. 4×4 Write and Hold Results

Transient simulation confirmed successful storage of all four target words.

The final array contents were:

Row	Cell States	Result
Row 0	X1 X2 X3 X4	1010
Row 1	X5 X6 X7 X8	0110
Row 2	X9 X10 X11 X12	1100
Row 3	X13 X14 X15 X16	0011

Each row retained its stored data after its corresponding wordline returned LOW.

Previously written rows also retained their states while later rows were being written. This demonstrated that inactive rows remained isolated from the active bitlines.

Figure 6. Row 0 transient simulation showing the stored value 1010.

[Insert X1–X4 waveform.]

Figure 7. Row 1 transient simulation showing the stored value 0110.

[Insert X5–X8 waveform.]

Figure 8. Row 2 transient simulation showing the stored value 1100.

[Insert X9–X12 waveform.]

Figure 9. Row 3 transient simulation showing the stored value 0011.

[Insert X13–X16 waveform.]

These results demonstrate:

- Four-bit parallel writing
 - Independent row selection
 - Data retention
 - Row isolation
 - Scaling from a single SRAM cell to a 16-cell memory array
-

9. Read Operation

The read operation was also examined to confirm that stored data could be accessed without intentionally overwriting the cell.

During SRAM read operation, the selected wordline connects the internal storage nodes to the differential bitlines. The stored state influences BL and BLB, allowing the stored bit to be detected externally.

In a practical SRAM macro, the resulting differential voltage is normally detected by a sense amplifier.

The simplified simulation used in this project focused on verifying the electrical response of the SRAM cell and array rather than implementing a complete commercial SRAM read path.

Figure 10. Representative SRAM read simulation.

[Insert the clearest read waveform from the completed experiments here.]

10. Relationship to Practical SRAM

The completed 4×4 array is a simplified version of the memory-cell matrix used in larger SRAM systems.

In a practical SRAM macro, the storage array would normally be surrounded by additional peripheral circuitry such as:

- Row decoder
- Column multiplexer
- Bitline precharge circuit
- Write driver
- Differential sense amplifier
- Timing and control logic

The basic data flow is:

Data Input

- Write Driver
- BL / BLB
- Selected SRAM Row
- Stored State

For reading:

Selected Wordline

- SRAM Cell
- BL / BLB Differential Signal
- Sense Amplifier
- Digital Data Output

Large SRAM arrays use the same fundamental cell organization but repeat the basic structure across many more rows and columns.

SRAM is commonly used in processor cache, embedded memories, local buffers, and other applications requiring fast access to temporary data.

11. Limitations

The project focused on understanding transistor-level SRAM operation and small-array organization rather than designing a production SRAM macro.

The following components were outside the scope of the final implementation:

- Full row decoder
- Column decoder and multiplexer
- Dedicated write driver
- Complete differential sense amplifier

- Full bitline precharge network
- Physical transistor layout
- DRC and LVS verification
- Process-voltage-temperature corner analysis
- Statistical mismatch analysis
- Large-array power and timing optimization

The MOSFET models used in LTspice were also simplified and were intended primarily for functional circuit-level study.

12. Future Work

Possible extensions of this project include:

- Designing a differential sense amplifier
- Adding realistic bitline precharge circuitry
- Designing a row address decoder
- Adding a write-driver circuit
- Measuring read delay and write delay
- Measuring static and dynamic power
- Performing static noise margin analysis
- Testing different transistor sizing ratios
- Performing process and temperature variation analysis
- Expanding the memory capacity
- Creating a transistor-level physical layout
- Performing DRC and LVS verification

A natural next step would be to move from LTspice functional simulation toward a CMOS/VLSI design environment and implement a small SRAM macro with peripheral circuits and physical layout.

13. Conclusion

This project demonstrated the development of a CMOS memory circuit from individual transistor behavior to a functional 4×4 SRAM array.

A transistor-level 6T SRAM cell was designed and simulated in LTspice. Write, hold, and read behavior were investigated before the cell was converted into a reusable hierarchical block.

The design was first expanded into a 2×2 array to verify shared wordlines, shared differential bitlines, multi-bit writing, and row isolation. The validated design was then expanded into a 4×4 array containing 16 SRAM cells.

The final array successfully stored four independent four-bit words:

- 1010
- 0110
- 1100
- 0011

The stored values remained stable after the corresponding wordlines were disabled and while other rows were being written.

Overall, the project demonstrated how CMOS transistor-level circuits can be organized into a functional memory architecture and provided practical experience with SRAM storage, hierarchical circuit design, differential bitlines, wordline-controlled access, and memory-array organization.